

CelebA Gender Classification

Binary image classification on the CelebA dataset: predict whether a celebrity photo is male or female. The challenge is limited data — 2,000 training, 1,000 validation, and 1,000 test images (218×178 px).

Approach: (1) train a CNN from scratch with data augmentation, (2) fast feature extraction with frozen VGG16, (3) fine-tune VGG16 on the classification task.

1. Environment Setup

Verify TensorFlow/Keras version.

```
In [ ]: import keras
        keras.__version__
```

```
Out[ ]: '3.10.0'
```

```
In [ ]: import os, shutil
```

2. Dataset Paths

Local train/validation/test folders with `male/` and `female/` subdirectories.

```
In [ ]: base_dir = './hw6'
        train_dir = os.path.join(base_dir, 'train')
        validation_dir = os.path.join(base_dir, 'validation')
        test_dir = os.path.join(base_dir, 'test')
        train_male_dir = os.path.join(train_dir, 'male')
        train_female_dir = os.path.join(train_dir, 'female')
        validation_male_dir = os.path.join(validation_dir, 'male')
        validation_female_dir = os.path.join(validation_dir, 'female')
        test_male_dir = os.path.join(test_dir, 'male')
        test_female_dir = os.path.join(test_dir, 'female')
```

Sanity Check

Confirm expected image counts in each split.

```
In [ ]: print('total training male images:', len(os.listdir(train_male_dir)))
        total training male images: 825
```

```
In [ ]: print('total training female images:', len(os.listdir(train_female_dir)))
        total training female images: 1175
```

```
In [ ]: print('total validation male images:', len(os.listdir(validation_male_dir)))
total validation male images: 413

In [ ]: print('total validation female images:', len(os.listdir(validation_female_dir)))
total validation female images: 587

In [ ]: print('total test male images:', len(os.listdir(test_male_dir)))
total test male images: 412

In [ ]: print('total test female images:', len(os.listdir(test_female_dir)))
total test female images: 588
```

3. Data Generators

Import libraries and count samples per split.

```
In [ ]: import os
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
train_count = len(os.listdir(train_male_dir)) + len(os.listdir(train_female_dir))
val_count = len(os.listdir(validation_male_dir)) + len(os.listdir(validation_female_dir))
test_count = len(os.listdir(test_male_dir)) + len(os.listdir(test_female_dir))
print(train_count, val_count, test_count)

2000 1000 1000
```

3.1 CNN from Scratch

4-block Conv2D network with dropout and heavy data augmentation.

```
In [ ]: img_height, img_width = 218, 178
batch_size = 32
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
test_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(img_height, img_width),
```

```

        batch_size=batch_size,
        class_mode='binary'
    )
validation_generator = test_datagen.flow_from_directory(
    validation_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)
model_scratch = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(img_height, img_width, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(512, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model_scratch.compile(
    loss='binary_crossentropy',
    optimizer=optimizers.RMSprop(learning_rate=1e-4),
    metrics=['accuracy']
)
history_scratch = model_scratch.fit(
    train_generator,
    epochs=30,
    validation_data=validation_generator
)
test_loss_s, test_acc_s = model_scratch.evaluate(test_generator)
print(f"Scratch CNN test accuracy: {test_acc_s:.3f}")

```

Found 2000 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
Epoch 1/30
63/63  **27s** 387ms/step - accuracy: 0.5718 - loss: 0.6805
- val_accuracy: 0.5870 - val_loss: 0.6637
Epoch 2/30
63/63  **21s** 339ms/step - accuracy: 0.5950 - loss: 0.6700
- val_accuracy: 0.6140 - val_loss: 0.6191
Epoch 3/30
63/63  **20s** 317ms/step - accuracy: 0.6323 - loss: 0.6382
- val_accuracy: 0.7410 - val_loss: 0.5554
Epoch 4/30
63/63  **21s** 339ms/step - accuracy: 0.6742 - loss: 0.6159
- val_accuracy: 0.7540 - val_loss: 0.5462
Epoch 5/30
63/63  **20s** 317ms/step - accuracy: 0.6560 - loss: 0.6256
- val_accuracy: 0.7620 - val_loss: 0.5365
Epoch 6/30
63/63  **22s** 344ms/step - accuracy: 0.6881 - loss: 0.5972
- val_accuracy: 0.7580 - val_loss: 0.5125
Epoch 7/30
63/63  **20s** 325ms/step - accuracy: 0.6834 - loss: 0.6101
- val_accuracy: 0.7590 - val_loss: 0.5074
Epoch 8/30
63/63  **21s** 323ms/step - accuracy: 0.6884 - loss: 0.5922
- val_accuracy: 0.7550 - val_loss: 0.5217
Epoch 9/30
63/63  **21s** 333ms/step - accuracy: 0.7095 - loss: 0.5725
- val_accuracy: 0.7730 - val_loss: 0.4892
Epoch 10/30
63/63  **20s** 319ms/step - accuracy: 0.7120 - loss: 0.5748
- val_accuracy: 0.7780 - val_loss: 0.4776
Epoch 11/30
63/63  **21s** 338ms/step - accuracy: 0.7212 - loss: 0.5676
- val_accuracy: 0.7880 - val_loss: 0.4808
Epoch 12/30
63/63  **20s** 322ms/step - accuracy: 0.7132 - loss: 0.5562
- val_accuracy: 0.7580 - val_loss: 0.5165
Epoch 13/30
63/63  **21s** 336ms/step - accuracy: 0.6873 - loss: 0.5825
- val_accuracy: 0.7890 - val_loss: 0.4853
Epoch 14/30
63/63  **23s** 373ms/step - accuracy: 0.7236 - loss: 0.5438
- val_accuracy: 0.7870 - val_loss: 0.4574
Epoch 15/30
63/63  **20s** 325ms/step - accuracy: 0.7188 - loss: 0.5625
- val_accuracy: 0.7950 - val_loss: 0.4637
Epoch 16/30
63/63  **22s** 342ms/step - accuracy: 0.7360 - loss: 0.5472
- val_accuracy: 0.7480 - val_loss: 0.4705
Epoch 17/30
63/63  **20s** 315ms/step - accuracy: 0.7326 - loss: 0.5447
- val_accuracy: 0.7960 - val_loss: 0.4448
Epoch 18/30
63/63  **21s** 338ms/step - accuracy: 0.7107 - loss: 0.5563

```

- val_accuracy: 0.7940 - val_loss: 0.4521
Epoch 19/30
63/63 ━━━━━━━━━━━ 20s 322ms/step - accuracy: 0.7239 - loss: 0.5381
- val_accuracy: 0.7960 - val_loss: 0.4504
Epoch 20/30
63/63 ━━━━━━━━━━━ 21s 332ms/step - accuracy: 0.7481 - loss: 0.5300
- val_accuracy: 0.7840 - val_loss: 0.4464
Epoch 21/30
63/63 ━━━━━━━━━━━ 22s 352ms/step - accuracy: 0.7504 - loss: 0.5189
- val_accuracy: 0.8010 - val_loss: 0.4245
Epoch 22/30
63/63 ━━━━━━━━━━━ 20s 316ms/step - accuracy: 0.7263 - loss: 0.5447
- val_accuracy: 0.7890 - val_loss: 0.4539
Epoch 23/30
63/63 ━━━━━━━━━━━ 21s 341ms/step - accuracy: 0.7248 - loss: 0.5482
- val_accuracy: 0.8050 - val_loss: 0.4255
Epoch 24/30
63/63 ━━━━━━━━━━━ 20s 316ms/step - accuracy: 0.7512 - loss: 0.5127
- val_accuracy: 0.8160 - val_loss: 0.4187
Epoch 25/30
63/63 ━━━━━━━━━━━ 21s 340ms/step - accuracy: 0.7553 - loss: 0.5089
- val_accuracy: 0.7390 - val_loss: 0.4852
Epoch 26/30
63/63 ━━━━━━━━━━━ 20s 325ms/step - accuracy: 0.7359 - loss: 0.5333
- val_accuracy: 0.8100 - val_loss: 0.4396
Epoch 27/30
63/63 ━━━━━━━━━━━ 42s 341ms/step - accuracy: 0.7468 - loss: 0.5162
- val_accuracy: 0.8040 - val_loss: 0.4093
Epoch 28/30
63/63 ━━━━━━━━━━━ 20s 321ms/step - accuracy: 0.7414 - loss: 0.5119
- val_accuracy: 0.7920 - val_loss: 0.4520
Epoch 29/30
63/63 ━━━━━━━━━━━ 22s 348ms/step - accuracy: 0.7274 - loss: 0.5343
- val_accuracy: 0.8040 - val_loss: 0.4012
Epoch 30/30
63/63 ━━━━━━━━━━━ 21s 336ms/step - accuracy: 0.7608 - loss: 0.5077
- val_accuracy: 0.7770 - val_loss: 0.4367
32/32 ━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.7958 - loss: 0.4203
Scratch CNN test accuracy: 0.780

```

3.2 Fast Feature Extraction (VGG16)

Extract bottleneck features with frozen VGG16, then train a dense classifier.

```

In [ ]: conv_base = VGG16(
        weights='imagenet',
        include_top=False,
        input_shape=(img_height, img_width, 3)
    )
conv_base.summary()
datagen = ImageDataGenerator(rescale=1./255)
batch_size = 32
def extract_features(directory, sample_count):
    generator = datagen.flow_from_directory(
        directory,

```

```

        target_size=(img_height, img_width),
        batch_size=batch_size,
        class_mode='binary',
        shuffle=False
    )
    feature_shape = conv_base.output_shape[1:]
    features = np.zeros((sample_count, *feature_shape), dtype='float32')
    labels = np.zeros(sample_count, dtype='float32')
    i = 0
    for inputs_batch, labels_batch in generator:
        features_batch = conv_base.predict(inputs_batch)
        bs = len(labels_batch)
        features[i:i+bs] = features_batch
        labels[i:i+bs] = labels_batch
        i += bs
    if i >= sample_count:
        break
    return features, labels
train_features, train_labels = extract_features(train_dir, train_count)
val_features, val_labels = extract_features(validation_dir, val_count)
test_features, test_labels = extract_features(test_dir, test_count)
train_features = train_features.reshape((train_count, -1))
val_features = val_features.reshape((val_count, -1))
test_features = test_features.reshape((test_count, -1))
feature_model = models.Sequential([
    layers.Dense(256, activation='relu', input_dim=train_features.shape[1]),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])
feature_model.compile(
    optimizer=optimizers.RMSprop(learning_rate=2e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
history_feat = feature_model.fit(
    train_features, train_labels,
    epochs=30,
    batch_size=32,
    validation_data=(val_features, val_labels)
)
test_loss_f, test_acc_f = feature_model.evaluate(test_features, test_labels)
print(f"Fast feature extraction test accuracy: {test_acc_f:.3f}")

```

Model: "vgg16"

Layer (type)	Output Shape	Par
input_layer_15 (InputLayer)	(None, 218, 178, 3)	
block1_conv1 (Conv2D)	(None, 218, 178, 64)	1
block1_conv2 (Conv2D)	(None, 218, 178, 64)	36
block1_pool (MaxPooling2D)	(None, 109, 89, 64)	
block2_conv1 (Conv2D)	(None, 109, 89, 128)	73
block2_conv2 (Conv2D)	(None, 109, 89, 128)	147
block2_pool (MaxPooling2D)	(None, 54, 44, 128)	
block3_conv1 (Conv2D)	(None, 54, 44, 256)	295
block3_conv2 (Conv2D)	(None, 54, 44, 256)	590
block3_conv3 (Conv2D)	(None, 54, 44, 256)	590
block3_pool (MaxPooling2D)	(None, 27, 22, 256)	
block4_conv1 (Conv2D)	(None, 27, 22, 512)	1,180
block4_conv2 (Conv2D)	(None, 27, 22, 512)	2,359
block4_conv3 (Conv2D)	(None, 27, 22, 512)	2,359
block4_pool (MaxPooling2D)	(None, 13, 11, 512)	
block5_conv1 (Conv2D)	(None, 13, 11, 512)	2,359
block5_conv2 (Conv2D)	(None, 13, 11, 512)	2,359
block5_conv3 (Conv2D)	(None, 13, 11, 512)	2,359
block5_pool (MaxPooling2D)	(None, 6, 5, 512)	

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

Found 2000 images belonging to 2 classes.

1/1		1s 752ms/step
1/1		0s 176ms/step
1/1		0s 172ms/step
1/1		0s 166ms/step
1/1		0s 165ms/step
1/1		0s 171ms/step
1/1		0s 170ms/step
1/1		0s 169ms/step
1/1		0s 167ms/step
1/1		0s 185ms/step
1/1		0s 198ms/step
1/1		0s 207ms/step
1/1		0s 241ms/step
1/1		0s 198ms/step
1/1		0s 164ms/step
1/1		0s 167ms/step
1/1		0s 166ms/step
1/1		0s 168ms/step
1/1		0s 167ms/step
1/1		0s 164ms/step
1/1		0s 165ms/step
1/1		0s 154ms/step
1/1		0s 166ms/step
1/1		0s 170ms/step
1/1		0s 167ms/step
1/1		0s 166ms/step
1/1		0s 166ms/step
1/1		0s 168ms/step
1/1		0s 167ms/step
1/1		0s 169ms/step
1/1		0s 168ms/step
1/1		0s 170ms/step
1/1		0s 168ms/step
1/1		0s 168ms/step
1/1		0s 181ms/step
1/1		0s 182ms/step
1/1		0s 178ms/step
1/1		0s 184ms/step
1/1		0s 176ms/step
1/1		0s 180ms/step
1/1		0s 186ms/step
1/1		0s 188ms/step
1/1		0s 180ms/step
1/1		0s 169ms/step
1/1		0s 172ms/step
1/1		0s 167ms/step
1/1		0s 167ms/step
1/1		0s 169ms/step
1/1		0s 165ms/step
1/1		0s 172ms/step
1/1		0s 168ms/step
1/1		0s 169ms/step
1/1		0s 174ms/step
1/1		0s 168ms/step
1/1		0s 170ms/step

```
1/1 _____ 0s 167ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 167ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 166ms/step
1/1 _____ 0s 176ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 1s 607ms/step
Found 1000 images belonging to 2 classes.
1/1 _____ 0s 173ms/step
1/1 _____ 0s 176ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 167ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 167ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 166ms/step
1/1 _____ 0s 171ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 177ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 178ms/step
1/1 _____ 0s 183ms/step
1/1 _____ 0s 183ms/step
1/1 _____ 0s 182ms/step
1/1 _____ 0s 183ms/step
1/1 _____ 0s 185ms/step
1/1 _____ 0s 182ms/step
1/1 _____ 0s 191ms/step
1/1 _____ 0s 182ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 424ms/step
Found 1000 images belonging to 2 classes.
1/1 _____ 0s 166ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 167ms/step
1/1 _____ 0s 144ms/step
1/1 _____ 0s 171ms/step
1/1 _____ 0s 171ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 170ms/step
```

```
1/1 _____ 0s 169ms/step
1/1 _____ 0s 167ms/step
1/1 _____ 0s 171ms/step
1/1 _____ 0s 171ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 173ms/step
1/1 _____ 0s 157ms/step
1/1 _____ 0s 173ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 177ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 170ms/step
1/1 _____ 0s 169ms/step
1/1 _____ 0s 168ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 172ms/step
1/1 _____ 0s 173ms/step
1/1 _____ 0s 61ms/step
Epoch 1/30
63/63 _____ 4s 41ms/step - accuracy: 0.7075 - loss: 0.7311 -
val_accuracy: 0.8550 - val_loss: 0.3371
Epoch 2/30
63/63 _____ 0s 5ms/step - accuracy: 0.8868 - loss: 0.2688 - v
al_accuracy: 0.8660 - val_loss: 0.2976
Epoch 3/30
63/63 _____ 0s 6ms/step - accuracy: 0.9179 - loss: 0.2048 - v
al_accuracy: 0.9100 - val_loss: 0.2082
Epoch 4/30
63/63 _____ 0s 5ms/step - accuracy: 0.9382 - loss: 0.1651 - v
al_accuracy: 0.9090 - val_loss: 0.2229
Epoch 5/30
63/63 _____ 0s 5ms/step - accuracy: 0.9208 - loss: 0.1800 - v
al_accuracy: 0.9210 - val_loss: 0.2107
Epoch 6/30
63/63 _____ 0s 5ms/step - accuracy: 0.9450 - loss: 0.1291 - v
al_accuracy: 0.9070 - val_loss: 0.2354
Epoch 7/30
63/63 _____ 1s 5ms/step - accuracy: 0.9607 - loss: 0.1015 - v
al_accuracy: 0.9250 - val_loss: 0.1958
Epoch 8/30
63/63 _____ 0s 5ms/step - accuracy: 0.9627 - loss: 0.0979 - v
al_accuracy: 0.9190 - val_loss: 0.2225
Epoch 9/30
63/63 _____ 0s 5ms/step - accuracy: 0.9652 - loss: 0.0820 - v
al_accuracy: 0.9130 - val_loss: 0.2374
Epoch 10/30
63/63 _____ 0s 5ms/step - accuracy: 0.9788 - loss: 0.0604 - v
al_accuracy: 0.8940 - val_loss: 0.2824
Epoch 11/30
63/63 _____ 0s 6ms/step - accuracy: 0.9661 - loss: 0.0790 - v
al_accuracy: 0.9280 - val_loss: 0.2126
Epoch 12/30
63/63 _____ 0s 5ms/step - accuracy: 0.9791 - loss: 0.0617 - v
al_accuracy: 0.9280 - val_loss: 0.2132
Epoch 13/30
63/63 _____ 0s 5ms/step - accuracy: 0.9838 - loss: 0.0484 - v
```

al_accuracy: 0.9030 - val_loss: 0.2636
Epoch 14/30
63/63  1s 5ms/step - accuracy: 0.9909 - loss: 0.0351 - v
al_accuracy: 0.9210 - val_loss: 0.2206
Epoch 15/30
63/63  0s 5ms/step - accuracy: 0.9897 - loss: 0.0380 - v
al_accuracy: 0.9220 - val_loss: 0.2445
Epoch 16/30
63/63  0s 5ms/step - accuracy: 0.9883 - loss: 0.0323 - v
al_accuracy: 0.9290 - val_loss: 0.2188
Epoch 17/30
63/63  0s 5ms/step - accuracy: 0.9900 - loss: 0.0313 - v
al_accuracy: 0.9050 - val_loss: 0.2861
Epoch 18/30
63/63  0s 5ms/step - accuracy: 0.9915 - loss: 0.0284 - v
al_accuracy: 0.9260 - val_loss: 0.2308
Epoch 19/30
63/63  0s 5ms/step - accuracy: 0.9924 - loss: 0.0206 - v
al_accuracy: 0.9260 - val_loss: 0.2448
Epoch 20/30
63/63  0s 5ms/step - accuracy: 0.9945 - loss: 0.0193 - v
al_accuracy: 0.9110 - val_loss: 0.2694
Epoch 21/30
63/63  0s 6ms/step - accuracy: 0.9983 - loss: 0.0169 - v
al_accuracy: 0.9150 - val_loss: 0.2725
Epoch 22/30
63/63  0s 5ms/step - accuracy: 0.9982 - loss: 0.0095 - v
al_accuracy: 0.9130 - val_loss: 0.2701
Epoch 23/30
63/63  0s 5ms/step - accuracy: 0.9963 - loss: 0.0136 - v
al_accuracy: 0.9290 - val_loss: 0.2744
Epoch 24/30
63/63  0s 5ms/step - accuracy: 0.9979 - loss: 0.0125 - v
al_accuracy: 0.9140 - val_loss: 0.3190
Epoch 25/30
63/63  0s 6ms/step - accuracy: 0.9941 - loss: 0.0158 - v
al_accuracy: 0.9240 - val_loss: 0.2642
Epoch 26/30
63/63  0s 5ms/step - accuracy: 0.9995 - loss: 0.0080 - v
al_accuracy: 0.9270 - val_loss: 0.2822
Epoch 27/30
63/63  0s 7ms/step - accuracy: 0.9980 - loss: 0.0115 - v
al_accuracy: 0.9220 - val_loss: 0.2667
Epoch 28/30
63/63  0s 7ms/step - accuracy: 0.9999 - loss: 0.0062 - v
al_accuracy: 0.9270 - val_loss: 0.2782
Epoch 29/30
63/63  0s 7ms/step - accuracy: 0.9970 - loss: 0.0108 - v
al_accuracy: 0.9030 - val_loss: 0.3731
Epoch 30/30
63/63  0s 7ms/step - accuracy: 1.0000 - loss: 0.0044 - v
al_accuracy: 0.9250 - val_loss: 0.2861
32/32  0s 4ms/step - accuracy: 0.9269 - loss: 0.2761
Fast feature extraction test accuracy: 0.916

3.3 Fine-Tuning — Phase 1

Add augmentation + dense head on frozen VGG16. Save best checkpoint by validation loss.

```
In [ ]: conv_base = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(img_height, img_width, 3)
)
conv_base.trainable = False
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
])
inputs = keras.Input(shape=(img_height, img_width, 3))
x = data_augmentation(inputs)
x = conv_base(x)
x = layers.Flatten()(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)
model = keras.Model(inputs, outputs)
model.compile(
    loss='binary_crossentropy',
    optimizer=optimizers.RMSprop(learning_rate=2e-4),
    metrics=['accuracy']
)
train_datagen_ft = ImageDataGenerator(rescale=1./255)
val_datagen_ft = ImageDataGenerator(rescale=1./255)
train_dataset = train_datagen_ft.flow_from_directory(
    train_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)
validation_dataset = val_datagen_ft.flow_from_directory(
    validation_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)
callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning.keras",
        save_best_only=True,
        monitor="val_loss"
    )
]
history_top = model.fit(
    train_dataset,
    epochs=10,
    validation_data=validation_dataset,
```

```
callbacks=callbacks
)
```

Found 2000 images belonging to 2 classes.

Found 1000 images belonging to 2 classes.

Epoch 1/10

63/63  **13s** 182ms/step - accuracy: 0.6888 - loss: 0.7794
- val_accuracy: 0.8670 - val_loss: 0.2999

Epoch 2/10

63/63  **11s** 177ms/step - accuracy: 0.8312 - loss: 0.3946
- val_accuracy: 0.8890 - val_loss: 0.2709

Epoch 3/10

63/63  **11s** 176ms/step - accuracy: 0.8606 - loss: 0.3229
- val_accuracy: 0.9100 - val_loss: 0.2248

Epoch 4/10

63/63  **11s** 169ms/step - accuracy: 0.8398 - loss: 0.3311
- val_accuracy: 0.9050 - val_loss: 0.2287

Epoch 5/10

63/63  **11s** 168ms/step - accuracy: 0.8605 - loss: 0.3287
- val_accuracy: 0.8780 - val_loss: 0.2754

Epoch 6/10

63/63  **11s** 173ms/step - accuracy: 0.8917 - loss: 0.2633
- val_accuracy: 0.9150 - val_loss: 0.2069

Epoch 7/10

63/63  **10s** 166ms/step - accuracy: 0.8965 - loss: 0.2555
- val_accuracy: 0.9160 - val_loss: 0.2095

Epoch 8/10

63/63  **11s** 173ms/step - accuracy: 0.8965 - loss: 0.2391
- val_accuracy: 0.9240 - val_loss: 0.1957

Epoch 9/10

63/63  **11s** 169ms/step - accuracy: 0.9083 - loss: 0.2187
- val_accuracy: 0.9070 - val_loss: 0.2269

Epoch 10/10

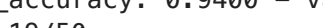
63/63  **11s** 175ms/step - accuracy: 0.9205 - loss: 0.2052
- val_accuracy: 0.9200 - val_loss: 0.1919

3.3 Fine-Tuning — Phase 2

Unfreeze from `block5_conv1` onward, train 50 epochs at learning rate 1e-5.

```
In [ ]: from tensorflow.keras.preprocessing.image import ImageDataGenerator
conv_base.trainable = True
set_trainable = False
for layer in conv_base.layers:
    if layer.name == "block5_conv1":
        set_trainable = True
    layer.trainable = set_trainable
model.compile(
    loss='binary_crossentropy',
    optimizer=optimizers.RMSprop(learning_rate=1e-5),
    metrics=['accuracy']
)
callbacks_ft = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning.keras",
```

```
        save_best_only=True,  
        monitor="val_accuracy",  
        mode="max"  
    )  
]  
history_ft = model.fit(  
    train_dataset,  
    epochs=50,  
    validation_data=validation_dataset,  
    callbacks=callbacks_ft  
)  
test_datagen = ImageDataGenerator(rescale=1./255)  
test_dataset = test_datagen.flow_from_directory(  
    test_dir,  
    target_size=(img_height, img_width),  
    batch_size=batch_size,  
    class_mode='binary'  
)
```

Epoch 1/50
63/63  **15s** 200ms/step - accuracy: 0.9045 - loss: 0.2243
- val_accuracy: 0.9290 - val_loss: 0.1789
Epoch 2/50
63/63  **12s** 194ms/step - accuracy: 0.9278 - loss: 0.1763
- val_accuracy: 0.9380 - val_loss: 0.1696
Epoch 3/50
63/63  **12s** 186ms/step - accuracy: 0.9202 - loss: 0.1674
- val_accuracy: 0.9370 - val_loss: 0.1649
Epoch 4/50
63/63  **12s** 186ms/step - accuracy: 0.9374 - loss: 0.1585
- val_accuracy: 0.9350 - val_loss: 0.1729
Epoch 5/50
63/63  **14s** 219ms/step - accuracy: 0.9416 - loss: 0.1412
- val_accuracy: 0.9430 - val_loss: 0.1631
Epoch 6/50
63/63  **12s** 185ms/step - accuracy: 0.9450 - loss: 0.1301
- val_accuracy: 0.9280 - val_loss: 0.1820
Epoch 7/50
63/63  **12s** 184ms/step - accuracy: 0.9650 - loss: 0.0915
- val_accuracy: 0.9410 - val_loss: 0.1606
Epoch 8/50
63/63  **12s** 185ms/step - accuracy: 0.9691 - loss: 0.0822
- val_accuracy: 0.9410 - val_loss: 0.1756
Epoch 9/50
63/63  **12s** 193ms/step - accuracy: 0.9639 - loss: 0.0884
- val_accuracy: 0.9460 - val_loss: 0.1556
Epoch 10/50
63/63  **12s** 186ms/step - accuracy: 0.9648 - loss: 0.0915
- val_accuracy: 0.9460 - val_loss: 0.1612
Epoch 11/50
63/63  **12s** 186ms/step - accuracy: 0.9651 - loss: 0.0875
- val_accuracy: 0.9450 - val_loss: 0.1633
Epoch 12/50
63/63  **12s** 193ms/step - accuracy: 0.9782 - loss: 0.0680
- val_accuracy: 0.9510 - val_loss: 0.1622
Epoch 13/50
63/63  **12s** 186ms/step - accuracy: 0.9746 - loss: 0.0648
- val_accuracy: 0.9270 - val_loss: 0.2302
Epoch 14/50
63/63  **12s** 186ms/step - accuracy: 0.9771 - loss: 0.0636
- val_accuracy: 0.9460 - val_loss: 0.1611
Epoch 15/50
63/63  **12s** 185ms/step - accuracy: 0.9758 - loss: 0.0665
- val_accuracy: 0.9470 - val_loss: 0.1702
Epoch 16/50
63/63  **12s** 185ms/step - accuracy: 0.9795 - loss: 0.0521
- val_accuracy: 0.9410 - val_loss: 0.1563
Epoch 17/50
63/63  **12s** 193ms/step - accuracy: 0.9805 - loss: 0.0542
- val_accuracy: 0.9520 - val_loss: 0.1668
Epoch 18/50
63/63  **12s** 187ms/step - accuracy: 0.9822 - loss: 0.0440
- val_accuracy: 0.9400 - val_loss: 0.1714
Epoch 19/50
63/63  **12s** 186ms/step - accuracy: 0.9804 - loss: 0.0529

- val_accuracy: 0.9430 - val_loss: 0.1685
Epoch 20/50
63/63  12s 186ms/step - accuracy: 0.9803 - loss: 0.0561
- val_accuracy: 0.9510 - val_loss: 0.1679
Epoch 21/50
63/63  12s 186ms/step - accuracy: 0.9778 - loss: 0.0497
- val_accuracy: 0.9410 - val_loss: 0.1816
Epoch 22/50
63/63  12s 186ms/step - accuracy: 0.9854 - loss: 0.0437
- val_accuracy: 0.9480 - val_loss: 0.1583
Epoch 23/50
63/63  12s 185ms/step - accuracy: 0.9849 - loss: 0.0353
- val_accuracy: 0.9510 - val_loss: 0.1630
Epoch 24/50
63/63  12s 184ms/step - accuracy: 0.9873 - loss: 0.0385
- val_accuracy: 0.9510 - val_loss: 0.1729
Epoch 25/50
63/63  12s 194ms/step - accuracy: 0.9874 - loss: 0.0394
- val_accuracy: 0.9530 - val_loss: 0.1593
Epoch 26/50
63/63  12s 193ms/step - accuracy: 0.9901 - loss: 0.0326
- val_accuracy: 0.9540 - val_loss: 0.1736
Epoch 27/50
63/63  12s 185ms/step - accuracy: 0.9932 - loss: 0.0239
- val_accuracy: 0.9480 - val_loss: 0.1908
Epoch 28/50
63/63  12s 186ms/step - accuracy: 0.9875 - loss: 0.0346
- val_accuracy: 0.9340 - val_loss: 0.2681
Epoch 29/50
63/63  12s 186ms/step - accuracy: 0.9926 - loss: 0.0280
- val_accuracy: 0.9490 - val_loss: 0.1654
Epoch 30/50
63/63  12s 186ms/step - accuracy: 0.9945 - loss: 0.0253
- val_accuracy: 0.9500 - val_loss: 0.1901
Epoch 31/50
63/63  12s 187ms/step - accuracy: 0.9906 - loss: 0.0274
- val_accuracy: 0.9530 - val_loss: 0.1716
Epoch 32/50
63/63  12s 185ms/step - accuracy: 0.9889 - loss: 0.0272
- val_accuracy: 0.9490 - val_loss: 0.2069
Epoch 33/50
63/63  12s 185ms/step - accuracy: 0.9941 - loss: 0.0230
- val_accuracy: 0.9470 - val_loss: 0.1708
Epoch 34/50
63/63  12s 185ms/step - accuracy: 0.9927 - loss: 0.0241
- val_accuracy: 0.9490 - val_loss: 0.2148
Epoch 35/50
63/63  12s 186ms/step - accuracy: 0.9951 - loss: 0.0164
- val_accuracy: 0.9380 - val_loss: 0.1922
Epoch 36/50
63/63  12s 186ms/step - accuracy: 0.9974 - loss: 0.0130
- val_accuracy: 0.9530 - val_loss: 0.1843
Epoch 37/50
63/63  12s 186ms/step - accuracy: 0.9945 - loss: 0.0146
- val_accuracy: 0.9530 - val_loss: 0.1884
Epoch 38/50

```

63/63 ————— 12s 187ms/step - accuracy: 0.9950 - loss: 0.0180
- val_accuracy: 0.9540 - val_loss: 0.1939
Epoch 39/50
63/63 ————— 12s 186ms/step - accuracy: 0.9883 - loss: 0.0263
- val_accuracy: 0.9540 - val_loss: 0.1905
Epoch 40/50
63/63 ————— 12s 186ms/step - accuracy: 0.9988 - loss: 0.0078
- val_accuracy: 0.9520 - val_loss: 0.1778
Epoch 41/50
63/63 ————— 12s 184ms/step - accuracy: 0.9944 - loss: 0.0180
- val_accuracy: 0.9490 - val_loss: 0.1871
Epoch 42/50
63/63 ————— 12s 185ms/step - accuracy: 0.9940 - loss: 0.0135
- val_accuracy: 0.9530 - val_loss: 0.2004
Epoch 43/50
63/63 ————— 12s 186ms/step - accuracy: 0.9963 - loss: 0.0183
- val_accuracy: 0.9490 - val_loss: 0.1861
Epoch 44/50
63/63 ————— 12s 186ms/step - accuracy: 0.9989 - loss: 0.0113
- val_accuracy: 0.9280 - val_loss: 0.2910
Epoch 45/50
63/63 ————— 12s 186ms/step - accuracy: 0.9962 - loss: 0.0135
- val_accuracy: 0.9490 - val_loss: 0.2475
Epoch 46/50
63/63 ————— 12s 186ms/step - accuracy: 0.9975 - loss: 0.0092
- val_accuracy: 0.9520 - val_loss: 0.1903
Epoch 47/50
63/63 ————— 12s 186ms/step - accuracy: 0.9963 - loss: 0.0177
- val_accuracy: 0.9510 - val_loss: 0.1803
Epoch 48/50
63/63 ————— 13s 212ms/step - accuracy: 0.9970 - loss: 0.0133
- val_accuracy: 0.9520 - val_loss: 0.2047
Epoch 49/50
63/63 ————— 12s 185ms/step - accuracy: 0.9968 - loss: 0.0093
- val_accuracy: 0.9530 - val_loss: 0.1999
Epoch 50/50
63/63 ————— 12s 185ms/step - accuracy: 0.9959 - loss: 0.0125
- val_accuracy: 0.9400 - val_loss: 0.2936
Found 1000 images belonging to 2 classes.

```

4. Test Set Evaluation

Load the best fine-tuned model and report final test accuracy.

```

In [ ]: model = keras.models.load_model("fine_tuning.keras")
        test_loss, test_acc = model.evaluate(test_dataset)
        print(f"Test accuracy: {test_acc:.3f}")

```

```

32/32 ————— 4s 107ms/step - accuracy: 0.9441 - loss: 0.2075
Test accuracy: 0.940

```